NanoQEC

Autoresearch for Neural Quantum Error Correction

A minimal AlphaQubit 2-style recurrent-transformer decoder
trained autonomously on Stim-generated surface code syndromes
via Hermes Agent on single-GPU compute

Zetetic Works Research Corporation
March 2026

INTERNAL TECHNICAL PLAN

0. Executive Summary

NanoQEC adapts the autoresearch pattern (Karpathy, 2026) to neural decoding for quantum error correction. An autonomous agent (Hermes Agent, Nous Research) iteratively modifies a minimal AlphaQubit 2-style recurrent-transformer decoder, trains it on Stim-generated surface code syndrome data under a fixed time budget, evaluates against a held-out validation set, and keeps or discards each change based on logical error rate improvement. The entire stack is open source, runs on a single GPU (M4 Pro MPS or a Prime Intellect A100/H100), and produces a research log of experiments while you sleep.

The decoder architecture is a deliberately simplified AQ2: interleaved lightweight gated-recurrence layers and spatial transformer layers operating on per-stabilizer representations, with temporal compression of syndrome frames and mean-pooled readout. Training data is generated on-the-fly from Stim's rotated surface code circuits with depolarizing noise. PyMatching provides the MWPM baseline. PyTorch is the training framework, with MPS backend for Apple Silicon and CUDA for cloud GPUs.

Hermes Agent drives the autonomous research loop via a custom nanoqec skill, replacing the program.md pattern from autoresearch with a richer skill document that includes syndrome-generation scripts, evaluation harness, and experiment logging. Hermes's persistent memory accumulates knowledge across experiment runs, and its cron scheduler enables overnight autonomous operation.

1. System Architecture

1.1 Information Flow

The system has four major subsystems. Hermes Agent orchestrates the loop, reading and modifying train.py, executing training runs via its terminal backend, and logging results. The data pipeline (prepare.py) generates syndrome data from Stim. The training engine (train.py) implements the neural decoder and optimization loop. The evaluation harness (eval.py) computes metrics and baselines.

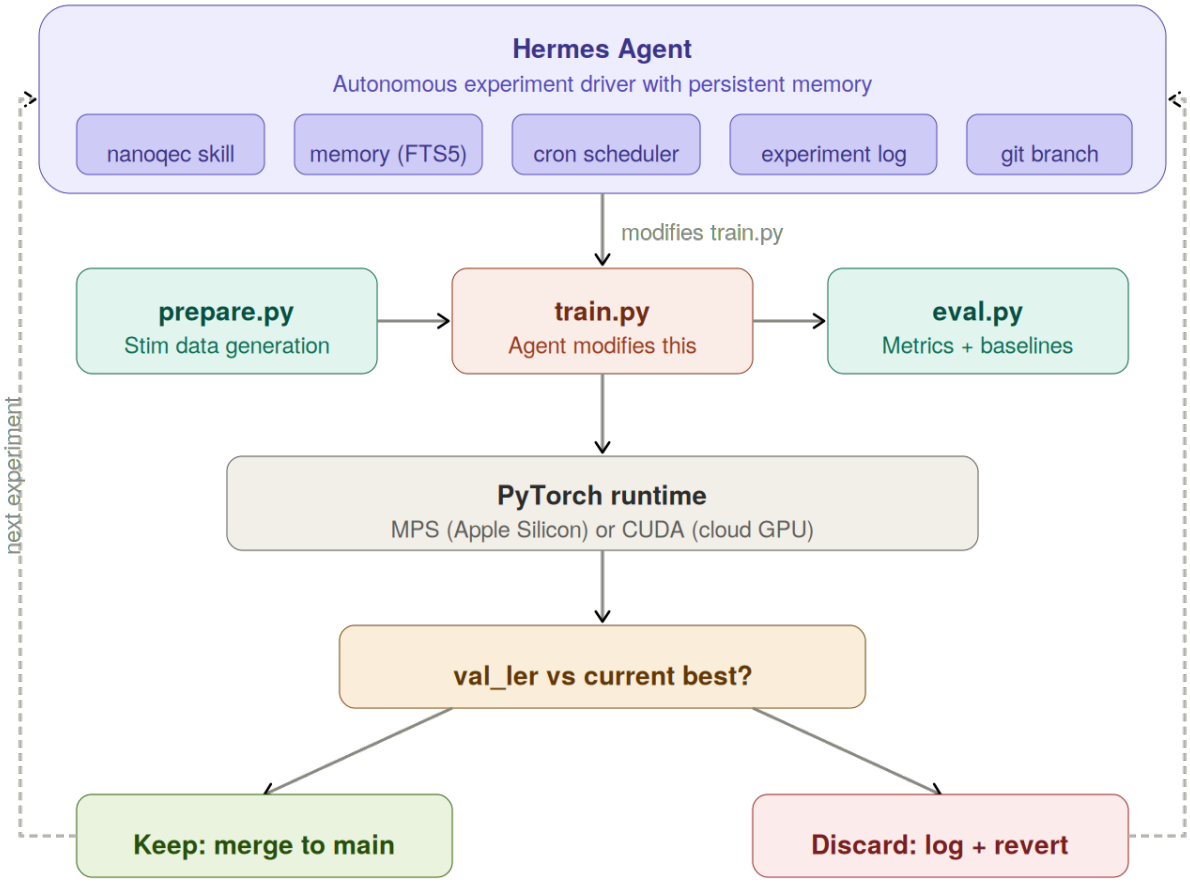

Figure 1: NanoQEC autoresearch loop and system architecture.

1.2 Autoresearch Loop

Each experiment cycle follows this sequence: (1) Hermes reads train.py and proposes a modification. (2) Hermes creates a git branch and applies the diff. (3) prepare.py generates or loads cached syndrome data from Stim. (4) train.py trains the decoder for a fixed wall-clock time budget. (5) eval.py evaluates logical error rate on validation data. (6) Hermes compares val_ler against the current best; if improved, the change is merged to main. (7) Hermes logs the experiment to experiments.jsonl. (8) Repeat.

1.3 Separation of Concerns

File	Owner	Role
prepare.py	Human (fixed)	Stim circuit generation, data loading, eval utilities
train.py	Agent (modified)	Model architecture, optimizer, training loop, hyperparams
eval.py	Human (fixed)	Evaluation harness, metric computation, baseline comparison
program.md	Human (iterated)	Autoresearch instructions for the agent
SKILL.md	Human (iterated)	Hermes Agent skill for the nanoqec research loop

2. Decoder Architecture (Minimal AQ2)

2.1 Design Principles

The minimal AQ2 decoder retains the core architectural innovations of AlphaQubit 2 while dramatically reducing scale for single-GPU training. It uses per-stabilizer representations, interleaved temporal (gated recurrence) and spatial (transformer attention) layers, temporal compression of consecutive syndrome frames, mean-pooled readout, and spatial rotary position embeddings encoding stabilizer grid coordinates.

2.2 Architecture Specification

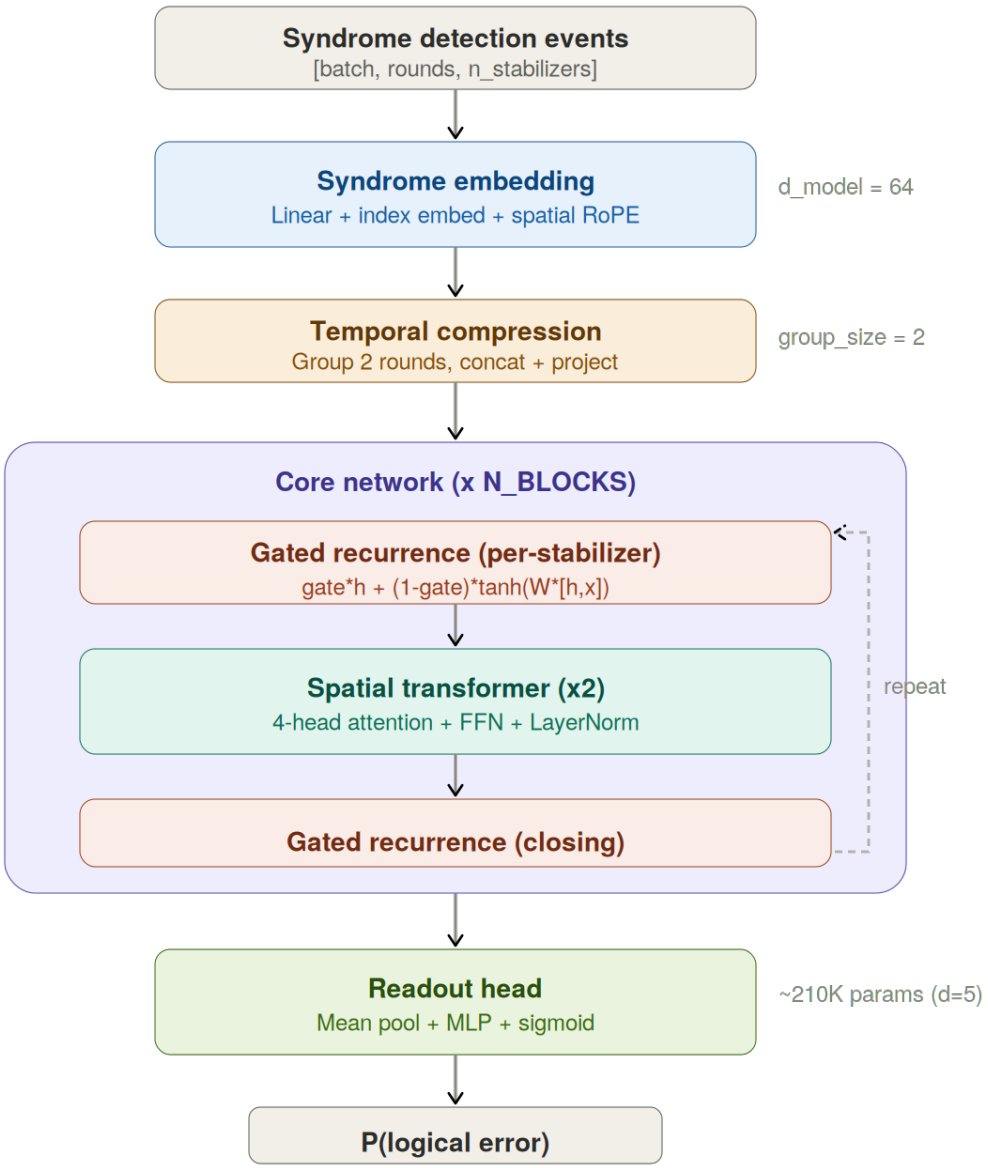

Figure 2: Minimal AQ2 decoder architecture.

2.3 Scaling Knobs for the Agent

These are the primary variables the agent should explore during autonomous research. Each experiment modifies one or a small number of these:

Knob	Default	Range	Notes
D_MODEL	64	32–256	Embedding dimension per stabilizer
N_BLOCKS	2	1–4	Number of (RNN + Transformer) blocks
N_HEADS	4	2–8	Attention heads
N_TX_PER_BLOCK	2	1–4	Transformer layers per block
GROUP_SIZE	2	1–4	Temporal compression factor
FFN_MULT	4	2–8	FFN hidden dim multiplier
LR	3e-4	1e-5 to 1e-2	Learning rate
BATCH_SIZE	256	64–2048	Training batch size
OPTIMIZER	AdamW	AdamW, Lion	Optimizer choice

3. Data Pipeline

3.1 Syndrome Generation with Stim

All training data is generated synthetically using Google's Stim simulator (Gidney, 2021). This is the same foundation used by the AlphaQubit papers. For the minimal setup, we use Stim's built-in depolarizing noise model applied to rotated surface code memory experiments. Stim's SIMD-vectorized sampling produces millions of syndrome shots per second on CPU, making on-the-fly data generation feasible even on laptops.

Stim Data Generation (prepare.py)

```
circuit = stim.Circuit.generated(
    "surface_code:rotated_memory_x",
    distance=d, rounds=d,
    after_clifford_depolarization=p,
    before_round_data_depolarization=p,
    after_reset_flip_probability=p,
    before_measure_flip_probability=p,
)
sampler = circuit.compile_detector_sampler()
detections, observables = sampler.sample(
    shots=n_shots, separate_observables=True
)
```

3.2 Data Configuration

Parameter	Value	Rationale
Distances	3, 5, (7)	d=3 for fast iteration, d=5 for signal, d=7 aspirational
Rounds	d (match distance)	Standard for memory experiments
Physical error rates	0.001–0.01 (8 values)	Sub-threshold to near-threshold
Training shots/config	50K–200K	Sufficient for small d convergence
Validation shots/config	10K	Held-out evaluation
Total training samples	~2–5M	Across all distances and error rates

3.3 Baseline: PyMatching (MWPM)

PyMatching provides the minimum-weight perfect matching baseline for comparison. The neural decoder's performance is measured as the ratio val_ler / mwpm_ler, where a ratio below 1.0 indicates the neural decoder outperforms MWPM. Pre-computed baselines are cached to avoid recomputation.

4. Training Setup

4.1 Fixed Time Budget

Following the autoresearch pattern, each experiment trains for a fixed wall-clock time budget. This makes experiments comparable regardless of what the agent changes. On M4 Pro (MPS), the default budget is 3 minutes. On Prime Intellect cloud GPUs, 5 minutes.

Platform	Time Budget	Throughput (est.)	Max Distance
M4 Pro (MPS)	3 min	~50K–200K samples/min	d=5 comfortable
Prime Intellect A100	5 min	~500K–1M samples/min	d=7 ideal
Prime Intellect H100	5 min	~1M–2M samples/min	d=7+ burst

4.2 Evaluation Metric

The primary metric is validation logical error rate (val_ler): the fraction of validation syndrome samples where the decoder incorrectly predicts the logical observable. Lower is better. The decoder's sigmoid output is thresholded at 0.5. Secondary metrics include per-distance val_ler, the MWPM comparison ratio, training throughput, and parameter count.

5. Hermes Agent Integration

5.1 Integration Architecture

Hermes Agent (Nous Research, Feb 2026) drives NanoQEC through three integration points: (1) A nanoqec skill in the agentskills.io format provides domain knowledge about QEC, the AQ2 architecture, and the experiment protocol. (2) Hermes executes training and evaluation via its terminal tool (local, Docker, or SSH to a Prime Intellect pod). (3) Hermes's built-in cron scheduler enables overnight autonomous operation with delivery of results via Telegram, Slack, or Discord.

5.2 Skill Structure

Skill Directory Layout

```
~/.hermes/skills/research/nanoqec/
  SKILL.md                 # Main instructions (program.md equivalent)
  references/
    ARCHITECTURE.md        # AQ2 architecture details
    QEC_PRIMER.md          # Surface code background
    EXPERIMENT_PROTOCOL.md # Step-by-step execution guide
  scripts/
    check_improvement.py   # Compare results against best
    plot_progress.py       # Generate progress charts
  assets/
    baseline_results.json  # Pre-computed MWPM baselines
```

5.3 Autonomous Operation

Hermes Agent's cron scheduler runs experiments overnight without human intervention. A typical overnight session (8 hours, 5-minute time budget + 5-minute overhead per experiment) yields approximately 48 experiments. Hermes's persistent memory system accumulates knowledge across runs: which architectural changes helped, platform-specific quirks, effective hyperparameter ranges, and common failure modes. This improves experiment quality over time without explicit programming.

5.4 Memory Accumulation

Unlike the stateless autoresearch pattern where the agent starts fresh each cycle, Hermes Agent's FTS5-backed memory and skill documents create a closed learning loop. When the agent discovers that (for example) increasing d_model beyond 128 causes MPS out-of-memory errors, it records this in a skill document and avoids it in future experiments. This is the key advantage of Hermes over a bare program.md approach.

6. Compute Strategy

6.1 Apple Silicon (M4 Pro, Primary Dev)

The M4 Pro's unified memory architecture and MPS backend provide a capable local development and experimentation platform. Key constraints: no flash attention kernel (use scaled_dot_product_attention with enable_math=True fallback), limited to d=5 comfortably within the time budget, and batch sizes capped at 128–256 depending on model size. Set PYTORCH_MPS_HIGH_WATERMARK_RATIO=0.0 to prevent memory fragmentation.

6.2 Cloud GPU: Prime Intellect (Primary Platform)

Prime Intellect is a compute exchange that aggregates GPU resources from 12+ cloud providers into a unified marketplace with competitive, transparent pricing. Their mission to democratize compute aligns with Zetetic Works' open-source neolab ethos. Prime Intellect also builds open-source distributed training infrastructure (PRIME-RL, OpenDiLoCo) and trains open models (INTELLECT-1/2/3), making them a natural partner for open AI-for-science work.

GPU (via Prime Intellect)	VRAM	Cost/hr	NanoQEC Fit
RTX 4090	24GB	~$0.32	Excellent for d=3,5 experiments
A6000	48GB	~$0.41	Good headroom for d=7
A100 80GB	80GB	~$0.79	Ideal for d=7+ and large batch sizes
H100 80GB	80GB	~$1.49	Maximum throughput for burst experiments

The A100 80GB at ~$0.79/hr is the recommended sweet spot. The ~210K parameter decoder is small enough that the A100 is never compute-bottlenecked, and 80GB VRAM allows very large batch sizes (2048+) and data caching entirely in GPU memory. For budget-conscious overnight runs, the RTX 4090 at ~$0.32/hr handles d=3,5 comfortably. For burst experiments pushing to d=7+, the H100 at ~$1.49/hr provides maximum throughput. Hermes Agent connects to Prime Intellect pods via SSH terminal backend. Total overnight cost: $0 (local M4 Pro) to ~$6 (8 hours on Prime Intellect A100).

7. Project Structure

Repository Layout

```
nanoqec/
  pyproject.toml       # Deps: torch, stim, pymatching, numpy, matplotlib
  .python-version      # 3.11
  prepare.py           # Data generation + eval utilities (DO NOT MODIFY)
  train.py             # Model + training loop (AGENT MODIFIES THIS)
  eval.py              # Evaluation harness (DO NOT MODIFY)
  program.md           # Autoresearch instructions (human-iterated)
  experiments.jsonl    # Experiment log (append-only)
  data/                # Cached syndrome data (.npz files)
  checkpoints/         # Saved model weights
  results/             # Evaluation results and plots
  .hermes/context.md   # Project-specific Hermes context
```

8. Implementation Plan

Working backward from the goal of autonomous overnight experiments:

Phase 0: Environment Setup (1–2 hours)

- Install Hermes Agent (curl installer)
- Configure Hermes with LLM backend (Claude Sonnet 4.6 via OpenRouter recommended)
- Install Prime Intellect CLI: uv tool install prime; prime login
- Create nanoqec/ repo with uv init; install deps (torch, stim, pymatching, numpy, matplotlib)
- Verify Stim and PyTorch work on target device (MPS locally; CUDA on Prime Intellect pod)
- Install nanoqec skill to ~/.hermes/skills/research/nanoqec/

Phase 1: Data Pipeline (2–3 hours)

- Implement prepare.py: Stim circuit gen, data caching, DataLoader, MWPM baselines
- Detector coordinate extraction for reshaping flat arrays to stabilizer grid
- Verify: generate 100K samples at d=3, p=0.005; confirm shapes and label distribution

Phase 2: Baseline Decoder (3–4 hours)

- Implement train.py: SyndromeEmbedding, GatedRecurrence, SpatialTransformer, TemporalCompression, ReadoutHead, NanoQECDecoder
- Training loop with fixed time budget, metric logging, RESULT output
- Verify: train on d=3 for 1 minute; confirm loss decreases and val_ler is reported

Phase 3: Evaluation Harness (1–2 hours)

- Implement eval.py: load model, evaluate across distances/error rates, MWPM comparison, LER plots
- Verify: eval produces results and plots; compare neural decoder to MWPM at d=3

Phase 4: Autoresearch Loop (2–3 hours)

- Write program.md and full SKILL.md with reference documents
- Implement check_improvement.py and plot_progress.py scripts
- Set up git repo; test one full experiment cycle manually via Hermes

Phase 5: Autonomous Operation (1 hour + overnight)

- Configure Hermes cron for overnight experiments
- Run first overnight batch; review log and progress plot in the morning
- Iterate on SKILL.md based on observed agent behavior

Phase 6: Scale and Explore (ongoing)

- Extend to d=7 on Prime Intellect A100/H100
- Explore color code circuits, soft readout support
- Benchmark against published neural decoder results

9. Success Criteria

Milestone	Criterion	Timeline
M0: Runs	train.py completes one experiment without error	Day 1
M1: Learns	val_ler improves with training (loss decreases)	Day 1
M2: Beats MWPM @ d=3	mwpm_ratio < 1.0 at d=3, p=0.005	Day 2–3
M3: Autonomous	Hermes runs 10+ experiments unattended	Day 2–3
M4: Beats MWPM @ d=5	mwpm_ratio < 1.0 at d=5, p=0.005	Week 1–2
M5: Scaling	Successful training and eval at d=7	Week 2–4
M6: Insight	Agent discovers non-obvious improvement	Ongoing

10. Open Source Stack

Component	Tool	License	Role
Syndrome simulation	Stim (Google)	Apache 2.0	Generate training data
Baseline decoder	PyMatching	Apache 2.0	MWPM comparison
Training framework	PyTorch	BSD	Neural network training
Compute platform	Prime Intellect	Open-source tooling	GPU marketplace and training infra
Agent framework	Hermes Agent (Nous)	Apache 2.0	Autonomous experiment driver
Package manager	uv (Astral)	MIT	Dependency management
Version control	Git	GPL	Experiment branching
LLM backend	Any (via OpenRouter)	Various	Agent intelligence

11. Risks and Mitigations

Risk	Impact	Mitigation
MPS backend instability	Training crashes	Catch exceptions, retry with reduced batch; CPU fallback
Agent makes breaking changes	Lost progress	Git branching: main always has working version

Risk	Impact	Mitigation
Overfitting at small d	Misleading metrics	Track train/val gap; mixed-distance training
Stim noise not representative	Poor generalization	Multiple noise models; noise augmentation
Hermes skill too rigid	Wasted experiments	Iterate SKILL.md based on logs; keep priorities loose
Compute too limited for d=7	Can't reach M5	Focus on d=3,5 insights; scale to Prime Intellect A100/H100

Appendix A: Key References

1. Bausch et al. "A scalable and real-time neural decoder for topological quantum codes" (AlphaQubit 2). arXiv:2512.07737, Dec 2025.

2. Bausch et al. "Learning high-accuracy error decoding for quantum processors" (AlphaQubit 1). Nature 635, 834–840, Nov 2024.

3. Gidney. "Stim: a fast stabilizer circuit simulator." Quantum 5, 497, Jul 2021.

4. Higgott & Gidney. "Sparse Blossom: correcting a million errors per core second with minimum-weight matching" (PyMatching). Quantum 9, 1600, 2025.

5. Lee, Hur & Park. "Scalable Neural Decoders for Practical Real-Time Quantum Error Correction." arXiv:2510.22724, Oct 2025.

6. Karpathy. "autoresearch." GitHub, Mar 2026.

7. Nous Research. "Hermes Agent." GitHub, Feb 2026.

Appendix B: Glossary

Detection event: XOR of consecutive stabilizer measurements; 1 indicates a change (potential error).

Logical error rate (LER): Probability that the decoder incorrectly predicts the logical observable.

MWPM: Minimum-weight perfect matching, the standard classical decoder.

Surface code: A topological QEC code defined on a 2D grid of qubits.

Syndrome: The set of stabilizer measurement outcomes at a given time step.

Distance (d): The code distance; a distance-d code can correct floor((d-1)/2) errors.

RoPE: Rotary Position Embedding; position encoding via rotation matrices applied to query/key vectors.

Gated recurrence: An RNN variant using element-wise gating (similar to GRU but lighter).